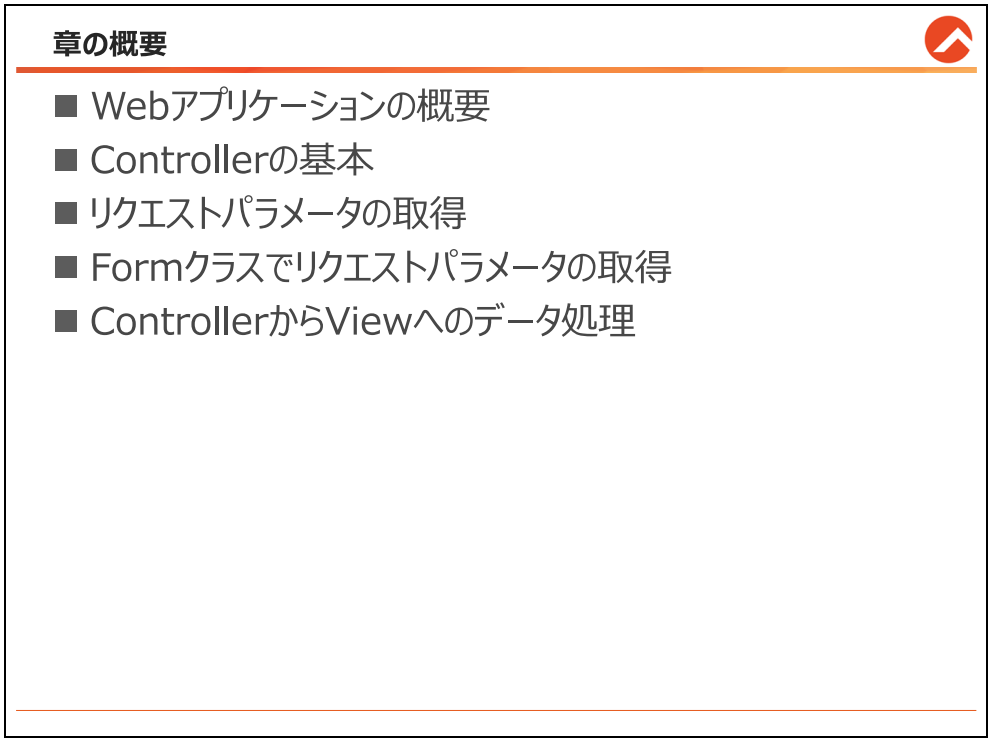
2

第2章 Spring Web アプリケーション

章の目標

- Web アプリケーションの基本的な流れを理解できる
- Controller を作成し、利用できる
- View から Controller へのリクエストパラメータを取得できる
- Form クラスを使用してリクエストパラメータを取得できる
- Model を通じて Controller から View にデータを表示できる

章の概要

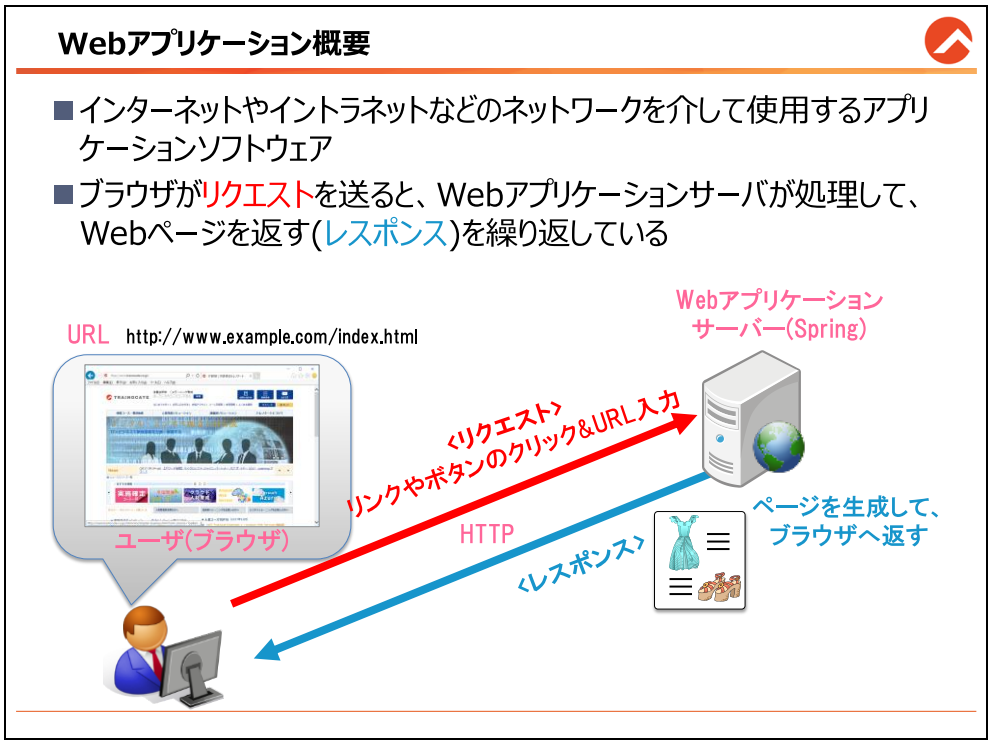


この章では、インターネットを介したアプリケーションの基本的な概念であるWeb アプリケーションについて学びます。

まず、Web アプリケーションの基本的な構造と機能について理解を深め、次に、SpringFramework を使用したWeb アプリケーション開発におけるController の基本について学習します。その後、Web アプリケーションにおけるリクエストパラメータの受け渡しについて学びます。最後に、「Thymeleaf」というテンプレートエンジンを使って、サーバー側で処理されたデータをWeb ページにどのように表示するかについても学びます。

この章を通して、Web アプリケーションの基礎的な概念とSpring Framework を使ったWebアプリケーション開発の基本を理解し、実際にWeb アプリケーションを構築するための基本的なスキルを身につけてみましょう。

2.1. Web アプリケーションの概要



Web アプリケーションとは？

Spring でWeb アプリケーションの作成を学ぶ前に、まずはWeb アプリケーションについてご説明します。

Web アプリケーションは、インターネットやイントラネットを介して使用されるソフトウェアです。日常的に使っているウェブサイト（例えば、ニュースサイトやオンラインショッピング、SNS など）は、実はすべてWeb アプリケーションの一部です。Web アプリケーションは、ブログや記事のようなWeb ページと異なり、ユーザの入力に応じて動的にコンテンツを変更したり、データベースと連携して情報を更新したりすることができます。

リクエストとレスポンスのプロセス

Web アプリケーションの基本的な動作は、**「リクエスト」と「レスポンス」**のやり取りによって成り立っています。ここでの「リクエスト」とは、皆様がブラウザを通じてサーバーに対して行う要求のことで、例えば**特定のページを表示**したい、あるいはフォームに**入力したデータを送信**したいといったことが含まれます。

一方で、「レスポンス」とはサーバーがリクエストに応じてブラウザに返す応答のことを指します。これは通常、HTML 形式のWeb ページとして表示され、ユーザに情報が送られます。

例えば、オンラインの服のショッピングサイトを訪れて、特定のブランドの服や靴を探しているとします。検索ボックスに「夏のファッション」と入力して検索ボタンをクリックします。これが「リクエスト」です。

このリクエストは、Web アプリケーションサーバーに送信され、Web アプリケーションはユーザが入力したブランド名に基づいて商品のデータベースを検索します。検索結果として、そのブランドの商品一覧が見つかります。この商品一覧がWeb ページとして整形され、ユーザのブラウザに送り返されます。これが「レスポンス」です。

その結果、のブラウザ上には求めていたブランドの商品一覧が表示されます。この一連のプロセスが、リクエストとレスポンスによって成り立っているのです。

Spring でのWeb アプリケーション開発

Spring Framework を利用すると、このリクエストとレスポンスのプロセスを簡単かつ効率的に扱うことが可能になります。Spring は記載したJava コードを基にして、ブラウザからのリクエストに適切に応答し、必要な処理を行い、適切なレスポンスを生成します。

この章では、Spring を使った基本的なWeb アプリケーションの構築方法について学びます。それでは、Spring を使ったWeb アプリケーション開発の魅力的な世界へ一緒に踏み込んでいきましょう。

2.1.1. 【 参考】サーブレットについて

【参考】サーブレットについて

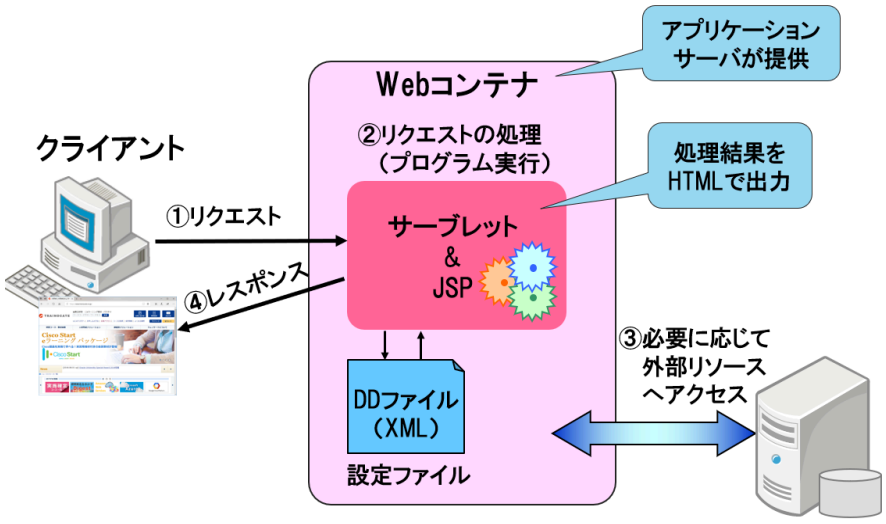
■ サーブレット
JavaのWeb開発でリクエスト受け取りとレスポンス送信を担当

■ 形
普通のJavaクラスに、Web関連のクラスを多数継承した形

■ 課題
Web関連機能を実装するためのコードの量が多く、冗長

■ Springでは？
サーブレットに代わりControllerクラスを採用。効率的な実装が可能

【参考】 サーブレットについて



サーブレットは、Java におけるWeb アプリケーション開発の基礎を提供する技術の一つです。サーブレットはWeb サーバー上で動作し、クライアントからのリクエストを受け取り、適切なレスポンスを生成して返す役割を持つクラスです。

【参考】サーブレットの基本

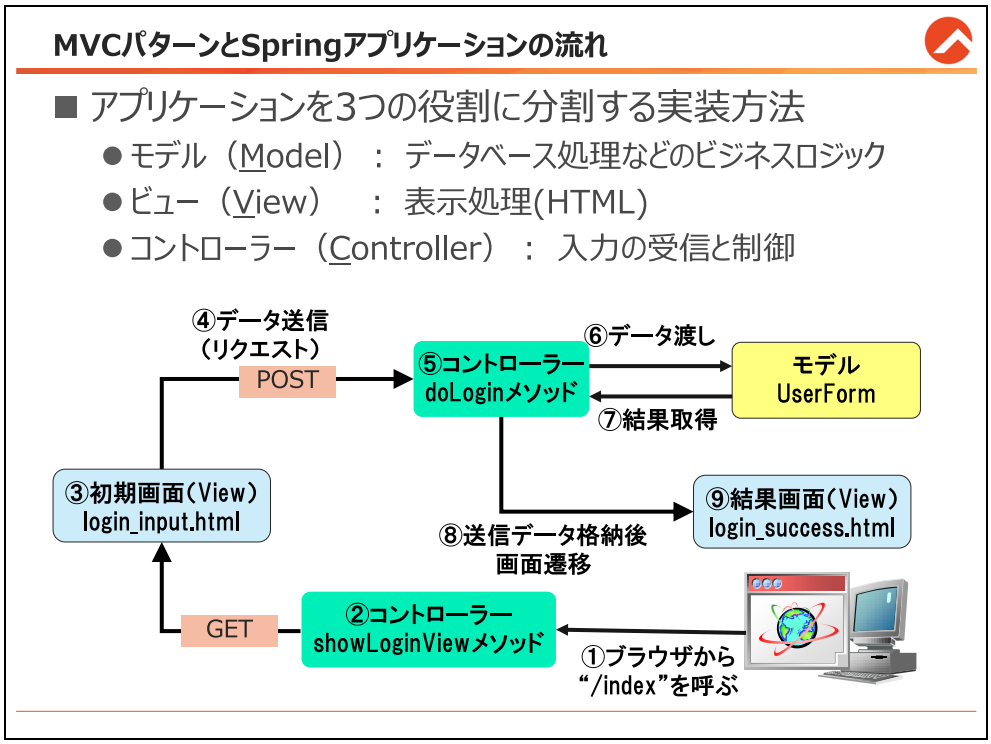
```
 1:import java.io.IOException;
 2:
 3:import jakarta.servlet.RequestDispatcher;
 4:import jakarta.servlet.ServletException;
 5:import jakarta.servlet.annotation.WebServlet;
 6:import jakarta.servlet.http.HttpServlet;
 7:import jakarta.servlet.http.HttpServletRequest;
 8:import jakarta.servlet.http.HttpServletResponse;
 9:
10:@WebServlet(urlPatterns = "/index") //URL マッピング
11:public class Sample1 extends HttpServlet { //HttpServlet の継承必
要
12:
13: //メソッド単位のURL マッピング不可
14: protected void doGet(HttpServletRequest request,
15: HttpServletResponse response)
16: throws ServletException, IOException {
17:
18: //index.jsp へ遷移のための長いコード
19: RequestDispatcher dispatcher =
20:
        request.getRequestDispatcher("/index.jsp");
21: dispatcher.forward(request, response);
22: }
23:}
```

このコードはサーブレットとSpring MVCの役割を比較する例です。Spring Boot 4.0.2ではServletの型は jakarta.servlet を使います。本章の配布アプリは、以降の @Controller とThymeleafで画面を作成します。

サーブレットの開発では、リクエストの処理やレスポンスの生成のために、比較的冗長なコードを書く必要がありました。サーブレットはWeb アプリケーションの中でコントローラーとしての役割を果たし、クライアントとサーバー間のデータのやり取りを管理します。

一方、Spring Framework では、**サーブレットクラスの代わりに**、よりシンプルで効率的な**Controller クラス**を提供しています。Spring のController クラスは、サーブレットの機能を簡潔に実装するためのアノテーションやツールを提供し、開発者がWeb アプリケーションのロジックにより集中できるように設計されています。

2.1.2.【 参考】MVC パターンと Spring アプリケーションの流れ



MVC パターンについて

この章では、複数のJava メソッドと、HTML ファイルを作成します。そして、「MVC パターン」と呼ばれる開発手法で実装を進めていきます。

MVC パターンでは、Model・View・Controller で役割を分割します。

このパターンは、明確に役割が分かれていて、保守性が高いプログラムを作ることができます。サーブレット/JSP の開発の際も多く使われていて、Spring では基本的にMVC パターンに基づいた実装する前提で設計されました。

Spring アプリケーションのMVC パターン

上の図には、Spring アプリケーションの典型的な流れが表現されています。

2章で扱うファイル名も載ってありますので、**2章の途中で必要であれば、随時このページに戻って、現在どこを説明しているかをご確認ください。**

それでは、次のページでコントローラーについてご説明します。

2.2. Controller の基本

Controllerの基本

```
@Controller
public class IndexController {
    @GetMapping("/index")
    public String index() {
        return "index";
```

■ Controllerクラス
・サーブレットのように、ユーザーからのリクエストを
受け取り、適切なレスポンスを生成するクラス

■ Controllerクラスの基本構文
・クラス名に@Controller アノテーションをつける
・メソッド名に@GetMappingをつけて、
URLとクラスのメソッドを関連付ける。
・returnには、表示したいhtmlファイル名を記載

■ ビューの作成
・Thymeleafというテンプレートエンジンを利用する

【参考】controller パッケージの作成

Spring でクラスを作成する際は、クラスに役割によって、異なるパッケージの中に作成をします。今からは、コントローラーの役割をする、IndexController クラスを作成する予定ですので、[controller]というパッケージを作成します。

パッケージの作成方法は、プロジェクトを展開し、src/main/java のパッケージを右クリック→[新規]→[パッケージ]→[名前]フォームに、**既存のパスを消さずに、[.controller]を追加すると作成されます。**



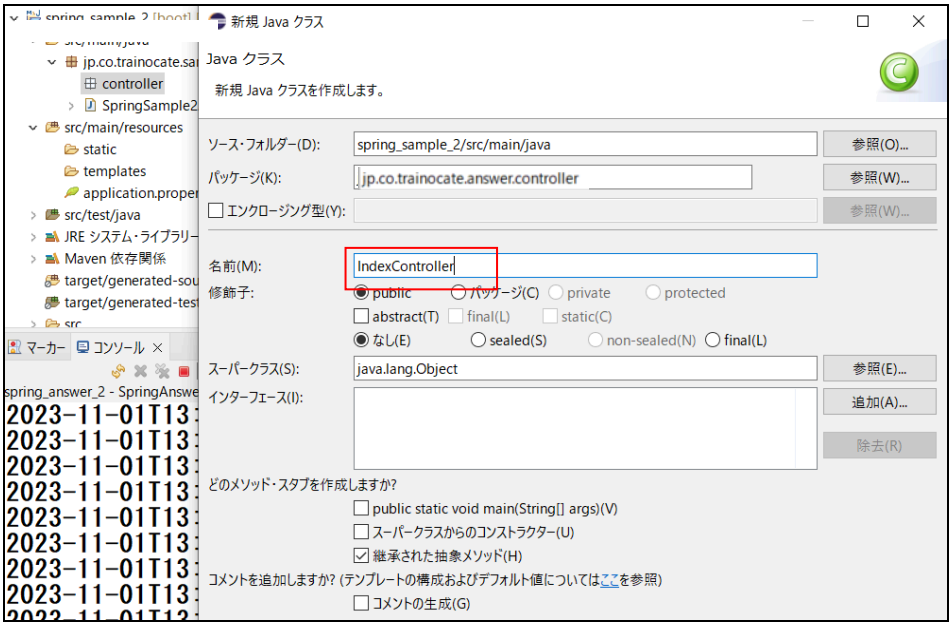
Spring Boot 4.0.2では、Web MVC用に spring-boot-starter-webmvc、MVCテスト用に spring-boot-starter-webmvc-test を使用します。配布POMは設定済みです。以降の @Controller・@GetMapping・Model の基本的な使い方は同じです。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
</dependency>
```

【参考】Controller クラス作成

Controller クラスを作成する際は、controller パッケージに、main メソッドを持たないクラスを作成します。

作成された[controller]パッケージを右クリック→[新規]→[クラス]→[名前]に「IndexController」と入力し、[完了]をクリック



…/controller/IndexController.java

【作業手順①】 Controller クラスの作成

```
 1:package jp.co.trainocate.answer.controller;
 2:
 3:import org.springframework.stereotype.Controller;
 4:import org.springframework.web.bind.annotation.GetMapping;
 5:
 6:@Controller
 7:public class IndexController {
 8: @GetMapping("/index")
 9: public String index() {
10: return "index";
11: }
12:}
```

上記のIndexControllerが、一番基本となるControllerの形です。前の節で確認した、サーブレットと比べて、とても簡略で短いコードになっています。

@Controller アノテーション

サーブレットで、Controller の役割をするためには、予めWeb 関連のクラスを継承するなど、冗長なコードを書かないといけませんでしたが、Spring では、**クラス名の上に、@Controller をつけるだけ**で、Controller としての役割を付与することができます。

```
6 @Controller
7 public class IndexController {
```

サーブレットのように、HttpServlet を継承して、HttpServletRequest とHttpServletResponse を設定する必要はありません。

リクエストマッピング

Controllerクラスには、プログラムの開始点となるmainメソッドはありません。では、Controllerのメソッドはいつどのように実行されるのでしょうか？

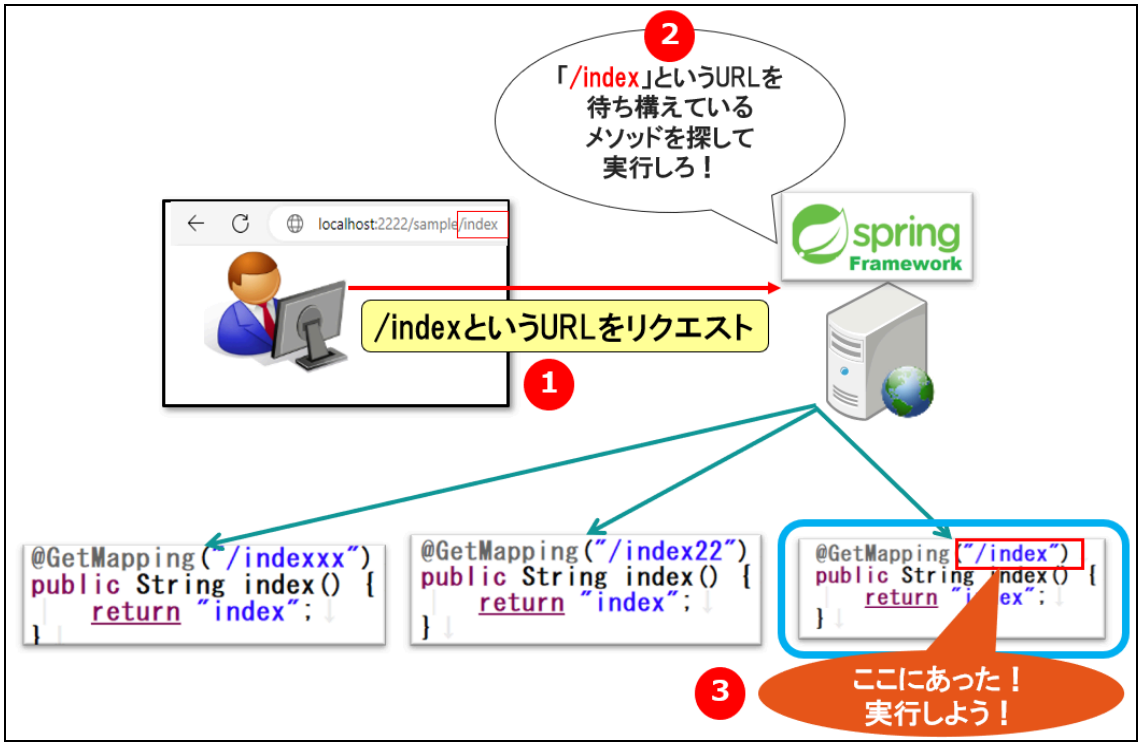
実は、Controller のメソッドは「**リクエストが来るまで待機**」しています。リクエストは、ユーザがWeb ページ上でリンクやボタンをクリックしたときにURL として送信されます。

このリクエストはSpring 環境のアプリケーションサーバーに届きます。サーバーは、登録されている全てのController を調べ、リクエストされたURL に対応するメソッドを探します。

URL を対応させる方法は、後述する「@GetMapping」アノテーションを、メソッド名の上につけるだけです。

```
8    @GetMapping("/index")
9    public String index() {
10       return "index";
11   }
```

対応するURL を見つけると、そのURL にマッピングされたController のメソッドが実行されます。今回の例では、ユーザーがブラウザで **http://ドメイン/index** にアクセスすると、このメソッドが呼び出されます。



【参考】 サーブレットでURL を対応させるコード

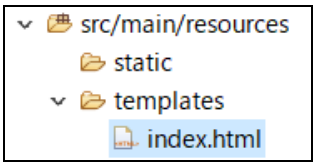
```
@WebServlet(urlPatterns = "/index")
public class Sample1 extends HttpServlet{
```

サーブレットの@WebServlet(urlPatterns)アノテーションと似たような役割をすると覚えると良いでしょう。ただ、サーブレットのURL 対応と決定的な異なる部分は、**サーブレットではクラス単位**でしたが、**Spring の@GetMapping では、メソッド単位でURL をマッピング**することができるということです。サーブレットでは、機能ごと1 つずつのクラスを作らないといけませんでしたが、Spring では1 つのクラス内に、複数の機能を実装できるので非常に便利です。

index メソッドの解説

```
 8    @GetMapping("/index")
 9    public String index() {
10        return "index";
11    }
```

- **9 行 public String index()**public は、このメソッドが外部からアクセス可能であることを意味します。String は、メソッドが文字列を返すことを示しています。index() はメソッド名で、このメソッドが何をするのかを表します。**Controller の中のメソッドは、通常は戻り値を String にします。**
- **10 行 return "index";**return には文字列を指定します。通常、この文字列は表示させたいhtml ファイル名を指定します。



Spring Boot では、ビュー(html)の名前を単純にreturn 文で返すだけで、適切なHTML 画面が表示されます。つまり、この場合は 「index.html」 という名前のページがユーザーに表示されます。**.html は省略しないといけません。**

【参考】 サーブレットでindex.jsp を表示するためには

```
RequestDispatcher dispatcher = request.getRequestDispatcher("/index.jsp");
dispatcher.forward(request, response);
```

今回のreturn は、サーブレットのrequest.getRequestDispatcher と似た役割をするコードだと覚えると良いでしょう。

Spring のreturn は、正確にはforward をしているのではなく、「指定したHTML ページを呼び出す」に近いですが、MVC パターンでの画面出力という観点では似ています。

Spring Boot の方がより簡潔で直感的なコードを書くことが可能になり、開発者の負担が軽減されます。

【作業手順②】ビューの作成と構成

Web アプリケーションにおいて、ユーザが見る画面を形作るファイルを「ビュー(View)」と称します。View は、HTML やJSP ファイルで構成されていることが一般的です。本書では、Java のWeb アプリケーション開発で広く用いられる「Thymeleaf（タイムリーフ）」というテンプレートエンジンを利用します。

テンプレートエンジンとは、HTML ファイルに数値や文字列などを動的に埋め込むことができる特別なライブラリのことです。このテンプレートエンジンを使用することにより、ユーザの名前をページに表示したり、商品一覧をリストアップしたりするような動的な処理が可能になります。

では、Controller のindex メソッドから遷移するビューをどのように作成するかをご紹介します。

…/resources/templates/index.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th="http://www.thymeleaf.org">
 3:<head>
 4: <meta charset="UTF-8" />
 5: <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
 6: <title>2 章 サンプル1</title>
 7:</head>
 8:
 9:<body>
10: <div>トップページ</div>
11:</body>
12:</html>
```

このビューの特徴として、2 行目に注目してください。ここでは、Spring Boot と相性が良いテンプレートエンジン「Thymeleaf」を使用するという宣言をしています。xmlns:th を記載することで、HTML ファイル内でThymeleaf の機能が利用可能になります。

[トップページ]と表示される部分（10 行目）からは、ユーザに見える画面の内容が入ります。今後、さまざまな情報やデザイン要素を追加していくことになります。

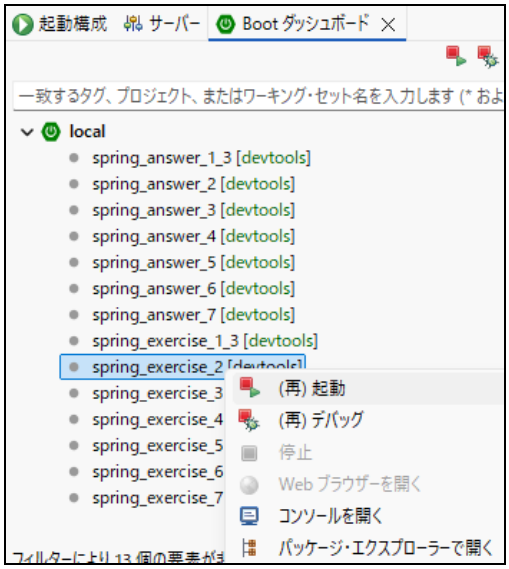
【作業手順③】Spring Boot アプリケーションの実行

IndexController のindex メソッドをブラウザからリクエストすることで、実行してみましょう。以下の手順に従ってください。

1. IndexController に誤字がないか確認し、保存します。
2. SpringExercise2Application.java のような main メソッドを持つ起動クラスを選択し、右クリックして[実行]→[Spring Bootアプリケーション]を選びます。



3. 実行構成の[JRE]がJDK 21であることを確認します。Bootダッシュボードを使う場合は、local の配下から対象プロジェクトを選んでも起動できます。
4. コンソールに起動完了のログが表示されるまで待ちます。再実行する場合は、先に動いている同じアプリケーションを停止してください。

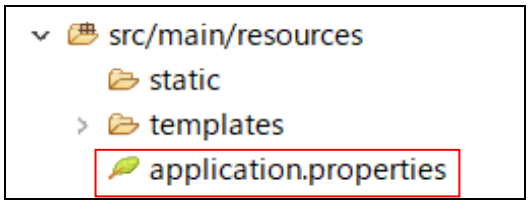


※ポップアップが出る際は、全て「許可」します。

5. 任意のブラウザ（Chrome、Edge など）のアドレスバーに以下のURL を入力しますhttp://localhost:8080/index
6. 以下のように、「トップページ」と表示されたら、実行成功です。



【参考】 application.properties ファイルについて




```
application.properties
1 server.servlet.context-path=/
2 server.port=8080
3
4 #サーバーを再起動しなくても、HTMLの修正が反映(ホットデプロイ)
5 spring.thymeleaf.cache=false
6
7 #その他データベース関連のコードは、3章で追記します
```

Spring Boot アプリケーションでは、application.properties ファイルが非常に重要です。このファイルは、アプリケーションの設定情報を保持し、アプリケーションの振る舞いをカスタマイズするために使用されます。application.properties ファイルは、プロジェクトの**src/main/resources ディレクトリ内に配置**されます。

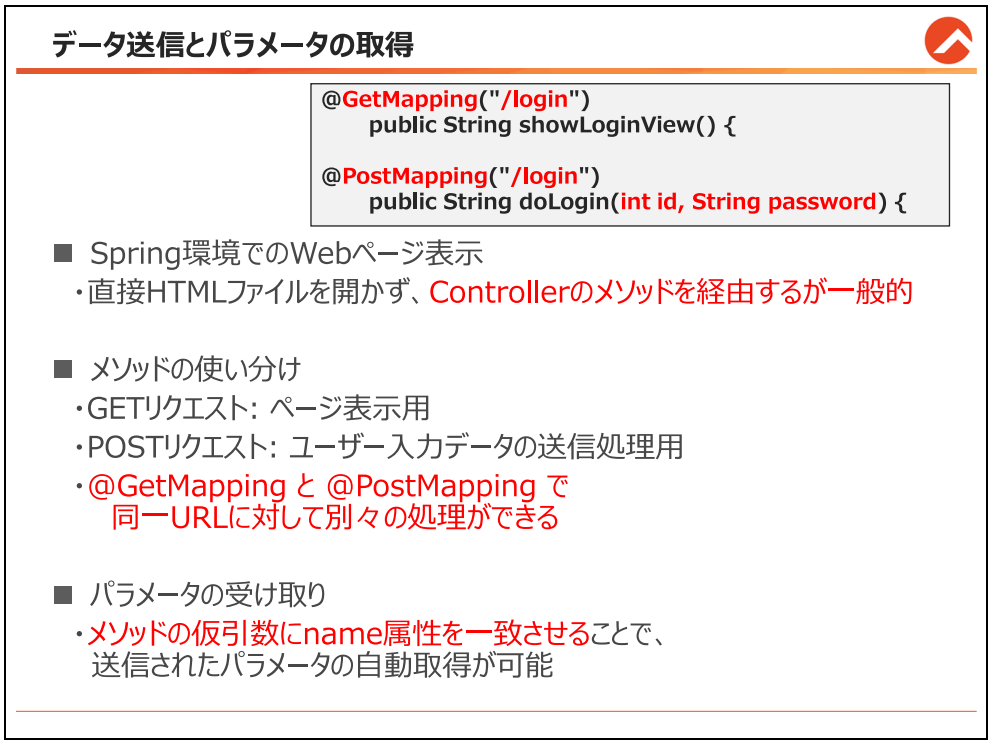
このファイルを修正すると、データベースの接続情報、サーバーのポート番号、コンテキストパスなど、さまざまな設定を行うことができます。特に開発時には、アプリケーションの動作環境に合わせて、これらの設定を調整することが一般的です。

以下は、application.properties ファイルに記載される可能性のある設定の例です。

- server.servlet.context-path=/sampleこの設定により、アプリケーションは /sample というコンテキストパスでアクセスされます。つまり、アプリケーションのURL の一部として / sample が含まれるようになります。
- server.port=2222この設定により、アプリケーションはポート番号 2222 でリッスンします。デフォルトのポート番号 8080 ではなく、別の番号を使用することで、他のアプリケーションとの衝突を避けることができます。
- spring.thymeleaf.cache=falseこの設定により、Thymeleaf テンプレートのキャッシュが無効になります。開発中には、テンプレートの変更が即座に反映されるようにするために便利です。

このように、application.properties ファイルを適切に設定することで、アプリケーションの振る舞いを柔軟に制御し、開発プロセスをスムーズに進めることができます。

2.3. データ送信とパラメータの取得



Web ページにおけるデータ送信の概要

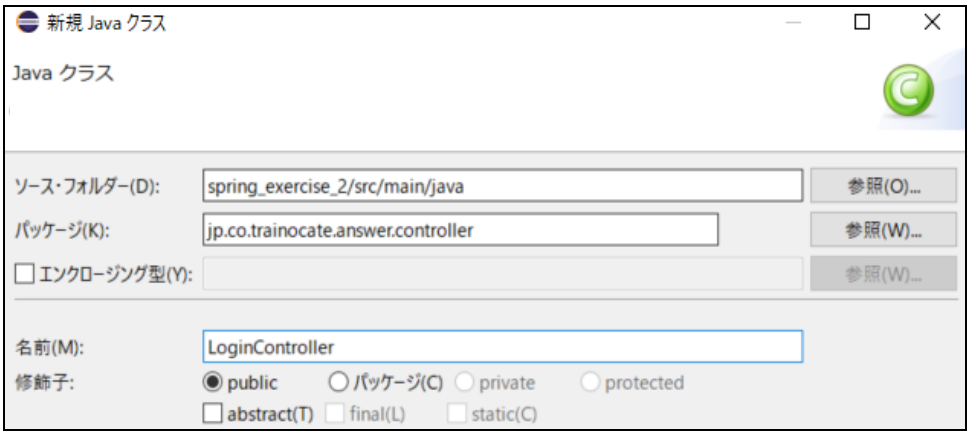
Web ページの大きな利点の一つは、ユーザがフォームにデータを入力して送信できる機能を持っている点です。以下のログインフォームの例のように、ユーザは情報を入力し、[ログイン]ボタンをクリックすることでそのデータをアプリケーションサーバーに送信することが可能です。



サーブレットでは、直接このページを開くことができましたが、**Spring では、直接HTML ページを開くのではなく、Controller のメソッドを経由させます。**よって、実装時は HTML ファイルより先に、画面遷移の処理のみを持つメソッドを作成する必要があります。

【作業手順①】ログイン画面を表示するメソッドの作成

まずは、ログイン処理用のController クラスを作成します。Spring では、画面ごとにControllerクラスを作成するのが一般的です。



従って、controller パッケージに、LoginController クラスを作成します。

…/controller/LoginController.java

```
 1:package jp.co.trainocate.answer.controller;
 2:
 3:import org.springframework.stereotype.Controller;
 4:import org.springframework.web.bind.annotation.GetMapping;
 5:
 6:@Controller
 7:public class LoginController {
 8:
 9: //Spring 環境では、html ページを直接より、
10: //遷移用のメソッドを経由する方法が一般的です
11:
12: @GetMapping("/login")
13: public String showLoginView() {
14: return "login_input";
15: }
```

[showLoginView()]は、ログインフォームのビュー（HTML ページ）を表示するためのメソッドです。このメソッドは処理を何もせず、単にView 名をreturn しています。**/login というURL がリクエストされたら、login_input.html をユーザへ表示してくれます。**

一般的に、Spring では、コントローラーのメソッドを経由してHTML ページを呼び出します。この手法により、URL とビューとの間に明確な対応関係を持たせ、アプリケーションの構造を整

理しやすくなります。また、必要に応じて追加のデータ処理やビジネスロジックをメソッド内に組み込むことが可能です。

【作業手順②】入力フォームがあるHTML の作成

…/resources/templates/login_input.html

```
 1:<!DOCTYPE html>
 2:<html xmlns:th="http://www.thymeleaf.org">
 3:<head>
 4:<meta charset="UTF-8"/>
 5:<meta name="viewport" content="width=device-width,
initial-scale=1.0"/>
 6:<title>Insert title here</title>
 7:</head>
 8:<body>
 9:<h1>ログイン画面</h1>

    <!--送信先を「/login」というURL に設定-->
10: <form th:action = "@{/login}" method="POST">
11:<p>社員番号 :<input type = "text" name = "id"/></p>
12:<p>パスワード:<input type = "password" name = "password"/></p>
13:<input type = "submit" value = "ログイン">
14:</form>
15:</body>
16:</html>
```

- **10 行目 <form th:action = “@{/login}” method = “POST”>**

<form>要素を用いてデータ送信のためのフォームを作成します。この要素のth:action 属性は、フォームデータが送信されるサーバー上のURL を指定できます。Thymeleaf では、ただのaction ではなく、**th:action** を使うことで、**実行環境に変化があってもURL を正しく解決**できます。**「@」は、アプリケーションのコンテキストパス(基本URL) を、自動生成**してくれます。URLが異なるサーバーで実行しても、影響がなくなる効果があります。ここでは/login がそのURL として指定されています。通常、**th:action 属性のURL 先は、@Controller のメソッドを指定**します。そして、method という属性がありますが、この属性は一般的なGET 方式の送信ではなく、POST 方式の送信をするという意味です。POST 方式にすると、パスワードがURL に表示されないなどのメリットがあり、隠したいデータを送信する時によく使われます。

- **11 行・12 行目 <input type = “text” name = “パラメータ名”>**

<input>要素はユーザがデータを入力するためのフィールドを提供します。type="text"を指定することでテキスト入力フィールドが、type="password"を指定することでパスワード入力フィールドがそれぞれ生成されます。ここで、このフォームのデータは、社員番号という日本語で認識されるのではなく、**name 属性の値で認識されます。今回は、「id」と「password」というnameでデータが送られるということを覚えてください。**

- **13 行目 <input type = "submit" value = "ログイン">**

<input type="submit">は送信ボタンを表します。このボタンをクリックすると、フォーム内のデータがth:action 属性で指定されたURL に送信されます。value 属性はボタン上に表示されるテキストを定義します。

では、送信したデータを、アプリケーションサーバー側では、どう受け取っているのでしょうか？ **アプリケーションサーバー側では、このフォームからの送信データを受け取る準備をしておく必要があります。**

【作業手順③】ログインデータの受信と処理

ユーザーがログインフォームに入力した情報をサーバー側で受け取るには、HTML フォームから送信されるデータを扱うためのコントローラーのメソッドを定義する必要があります。ここでは、ログインフォームのデータを処理するSpring のコントローラーメソッドの実装例を見ていきます。

…/controller/LoginController.java

```
 1://【省略】
 2:
 3:@GetMapping("/login")
 4:public String showLoginView() {
 5: return "login_input";
 6:}
 7:
 8://同じ「login」というURL でも、
 9://Get かPost かによって、異なるメソッドが実行されます
10:@PostMapping("/login")
11:public String doLogin(int id, String password) {
12: //仮引数を記載するだけで、自動的にデータが格納されます
13: System.out.println(id);
14: System.out.println(password);
15: return "index";
```

LoginController クラスのdoLogin メソッドは、ログインフォームから送信されたデータを受け取るために使用されます。

- **10 行目 @PostMapping("/login")**

@PostMapping は、POST リクエストが送られた際に、このメソッドが実行されることを示しています。3 行でも同じ/login というURL を待ち受けていますが、普通のGet 方式での呼び出しがあった場合は、showLoginView()メソッドが実行され、Post 方式の呼び出しがあった場合は、こちらのdoLogin メソッドが実行されます。

- **11 行目 仮引数 int id, String password**

login_input.html から送信したデータのname 属性に対応する値を受け取るために使用されています。**Spring では、Controller のメソッドの仮引数をhtml のname 属性の値と合わせるだけで、パラメータを受け取ることができます。**login_input.html では、「id」と「password」のデータが送られるので、引数名も、int id, String password に合わせます。

【参考】サーブレットでパラメータを受け取る方法

```
// 「id」という名前で送られてくるデータを取得し、整数に変換
int id = Integer.parseInt(request.getParameter("id"));

// 「password」という名前で送られてくるデータを取得
String password = request.getParameter("password");
```

サーブレットでは、上記のような冗長なコードが必要でしたが、Spring ではメソッドの引数に変数を入れておくだけで取得できるので、とても便利だというのがわかります。

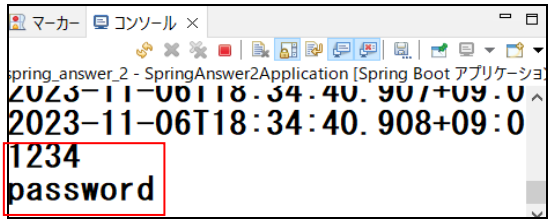
- **13 行、14 行目の出力**

本当にid とpassword のデータを取得できたのか、コンソールに出力することで、検証をしています。結果は、以下の画像のように、入力した値を問題なく取得できているのがわかります。

動作確認

http://localhost:8080/login

社員番号:1234パスワード:password

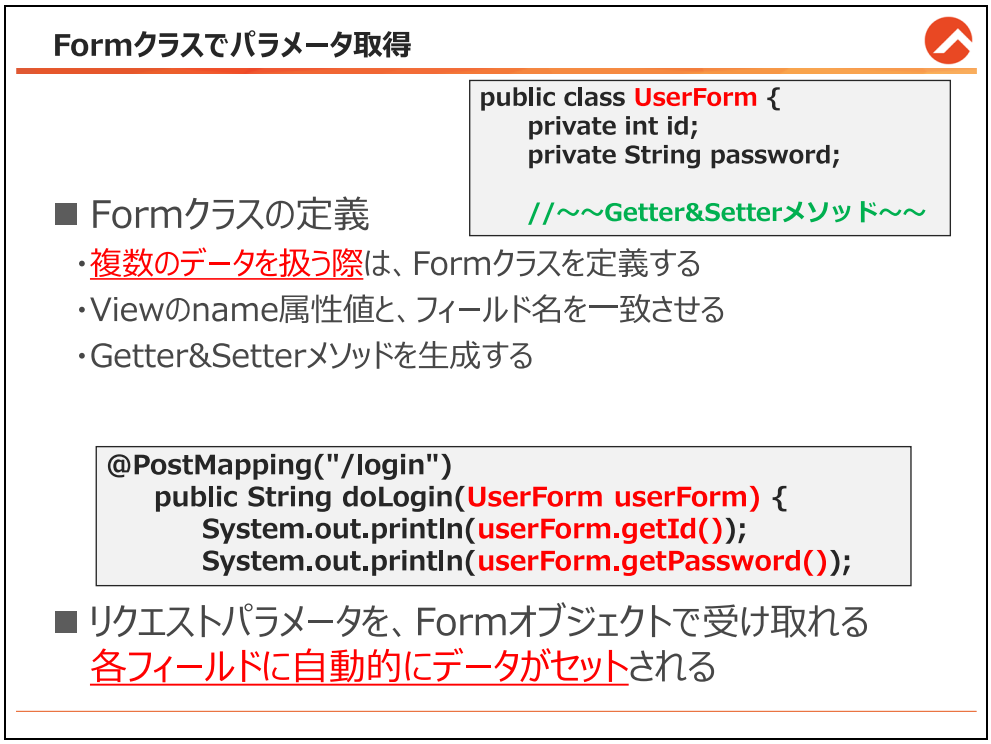


このように、Spring ではメソッドの仮引数を入力値の型に合わせて記載するだけで、自動的にリクエストパラメータからのデータの取得をしてくれます。

そして、同じURL に対して異なるHTTP メソッド（GET やPOST）を用いることで、異なる処理を行うメソッドを分けて定義することができます。これにより、フォームの表示とフォームからのデータ受信を別々のメソッドで扱うことができます。

Spring を使った開発は、このように「決まったルール通り」命名をすれば、後ろで自動的にやってくれるので、便利です。しかし、その反面、**1 文字でもルール通り命名しないと、思い通りに動かないこともありますので、変数名やメソッド名を命名する時は、注意が必要**です。

2.4. Form クラスでパラメータ取得



Form クラスを使用する目的

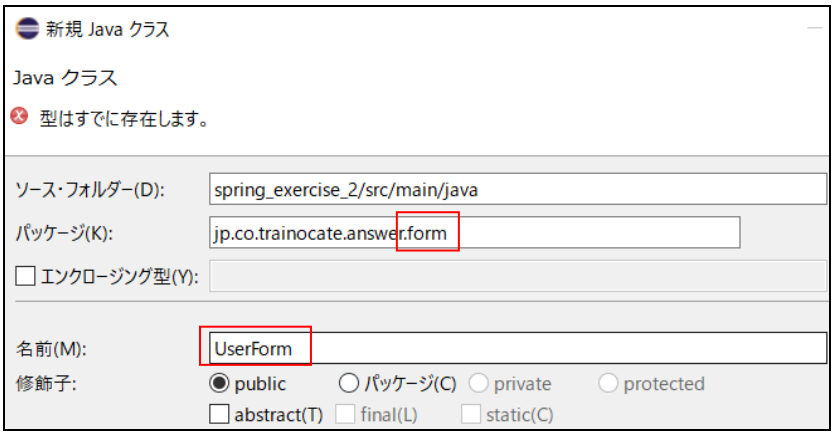
Web アプリケーションではユーザがフォームに複数の情報を入力する場面が頻繁にあります。例えば、登録フォームに名前、住所、電話番号など、多くのフィールドを記入する場合、これらの個別の情報を Controller のメソッドで一つずつ引数として取り扱うのは煩雑であり、コードの複雑さを増加させます。Controllerのメソッドの引数をまとめて管理するために、Formクラスを利用できます。

【作業手順①】 UserForm クラスの作成

まず、spring_exercise_2 を右クリックし、[form]というパッケージを作成します。



その後、[form]パッケージに、[UserForm]というクラスを作成します。



UserForm クラスには、**View から送られるデータをフィールドして記載し、Getter&Setter メソッドを定義**します。今回の例では、2 つのフィールドしか記載していませんが、複数のデータを扱うある場合は、全て記載します。

ここで注意点は、**フィールド名は、必ずView からのname 属性値と一致させる**ことです。

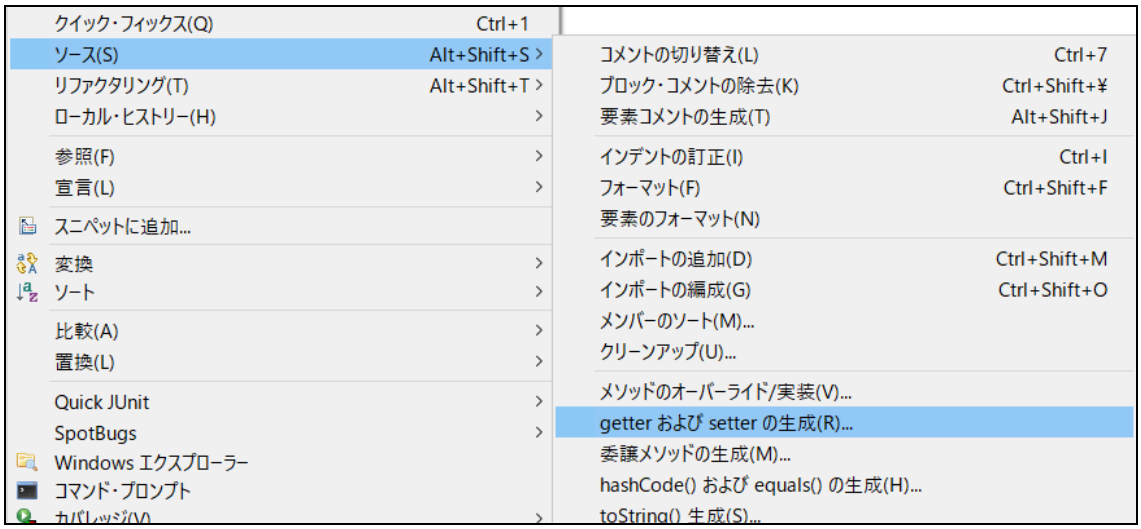
…/form/UserForm.java

```
 1:package jp.co.trainocate.answer.form;
 2:
 3:public class UserForm {
 4: private int id;
 5: private String password;
 6:
 7: public int getId() {
 8: return id;
 9: }
10: public void setId(int id) {
11: this.id = id;
12: }
13: public String getPassword() {
14: return password;
15: }
16: public void setPassword(String password) {
17: this.password = password;
18: }
19:}
```

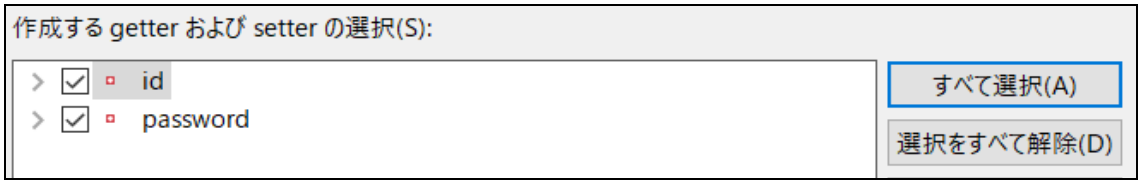
【参考】 Getter&Setter メソッドをEclipse で簡単に生成する方法

Getter&Setter メソッドは、定型文ですので、Eclipse で簡単に生成する機能があります。

Form クラス内のフィールドの定義が終わったら、以下の手順に従ってください。[右クリック]→[ソース]→[Getter および Setter の生成]



フィールドにチェックを入れて、[生成]をクリックすると、メソッドが生成されます。



【作業手順②】Form クラスのオブジェクトでパラメータを取得

それでは、この Form クラスを利用して、複数のリクエストパラメータを受け取るように、既存のLoginController を修正します。(サンプルは、LoginController2)

…/controller/LoginController2.java

```
1://【省略】
2:
3:@PostMapping("/login2")
4: public String doLogin(UserForm userForm) {
5: //仮引数にForm クラスを記載するだけで、
6: //自動的にデータがフィールドにセットされます
7: System.out.println(userForm.getId());
8: System.out.println(userForm.getPassword());
9: return "index";
10: }
```

- **4 行目 doLogin(UserForm userForm)**

doLogin メソッドの仮引数に、UserForm のオブジェクトを入れています。Spring では、**仮引数にForm クラスのオブジェクトを入れるだけで、リクエストパラメータがUserForm オブジェクトの対応するフィールド(id, password)に自動的にセットされます。**よって、各リクエストパラメータを個別に取得するためのコードを書く必要がなくなります。

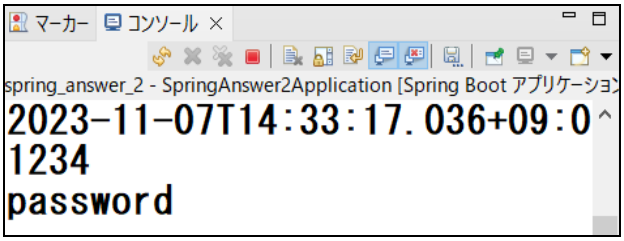
- **7-8 行目**

UserForm クラスに予め作成しておいた、getId()とgetPassword()メソッドを呼び出すことで、userForm オブジェクトにバインドされたID とパスワードを取得し、コンソールに出力します。

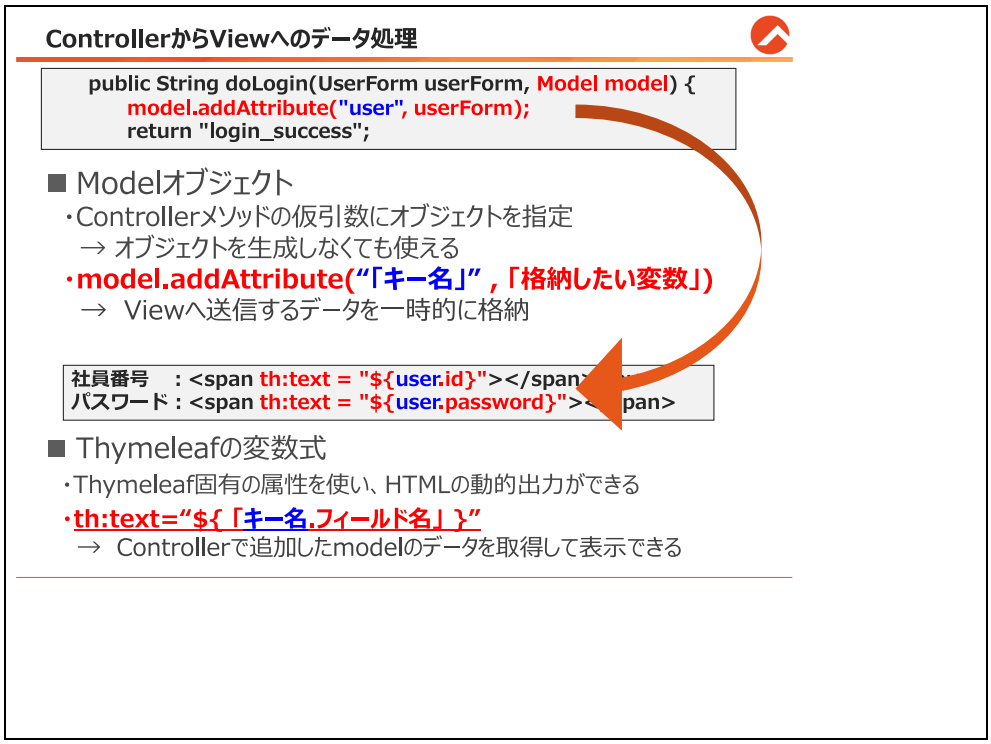
動作確認

以下のURL から、動作確認をすると、同じようにView からのリクエストパラメータを取得してコンソール上に出力できているのがわかります。

http://localhost:8080/login2



2.5. Controller から View へのデータ処理



前の節では、View からのリクエストパラメータを、Controller で受け取る方法についてご説明しました。

この節では、逆にController からView へデータを送り、画面上に表示する方法についてご説明します。

【作業手順③】 model オブジェクトにデータを格納

Spring Controller から、View へデータを送るためには、サーブレットと同じくリクエストスコープを利用します。リクエストスコープは、別の場所にデータを送るために、**一時的にデータを入れておく箱**だと思ってください。

…/controller/LoginController3.java

```
 1://省略
 2:import org.springframework.ui.Model;
 3:
 4:@Controller
 5:public class LoginController3 {
 6:
 7: @GetMapping("/login3")
 8: public String showLoginView() {
 9: return "login_input3";
10: }
11:
12: @PostMapping("/login3")
13: public String doLogin(UserForm userForm, Model model) {
14: //リクエストスコープに、userForm のデータを格納
15: model.addAttribute("user", userForm);
16:
17: return "login_success";
18: }
19:}
```

Spring フレームワークでは、Model オブジェクトを使用して、Controller からView にデータを渡します。**Modelオブジェクトの addAttribute メソッドを使うと、Model という箱に、データを一時保管できます**。サーブレットのrequest.setAttribute()に相当しますが、より洗練された方法です。

- **13 行目 doLogin(UserForm userForm, Model model)**

UserForm オブジェクトは、フォームから送信されたデータを受け取り、リクエストスコープのオブジェクト(データ一時保管用の箱)を生み出すため、Model オブジェクトをメソッドの仮引数に入れます。

UserForm の時と同様に、**Spring ではメソッドの仮引数にオブジェクト名を書くと、オブジェクトを生成しなくても、使うことができます。**

- **14-15 行目 model.addAttribute("user", userForm);**

この行は、UserForm オブジェクトを"user"というキーでモデルに追加しています。第1 引数:キー名第2 引数:格納したい変数

これにより、return で呼び出すlogin_success.html でも、userForm オブジェクトのデータが使用可能になります。

【参考】 サーブレットでリクエストデータを格納するコード

```
request.setAttribute("id", id);
request.setAttribute("password", password);
```

サーブレットのrequest.setAttribute と役割も、使い方は同じですので、参考にしてください。

- **17 行 return "login_success";**

モデルに格納されたデータと共に、login_success.html を呼び出します。

このようにModel オブジェクトを使うと、Controller からView へデータを「一時的に入れておく箱」としてリクエストスコープを利用することができ、Spring のMVC パターンに沿った方法でデータを渡すことができます。

【作業手順④】 Thymeleaf を利用したデータ出力

Thymeleaf は、Spring Framework と組み合わせて使用されるモダンなサーバーサイドJava テンプレートエンジンです。**Thymeleaf を利用することで、Controller から送信されたデータをHTML 内で動的に表示することができます。**

…/resources/templates/login_success.html

```
1:<!DOCTYPE html>
2:<html xmlns:th="http://www.thymeleaf.org">
3:<head>
4:<meta charset="UTF-8"/>
5:<meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
6:<title>2 章 ログイン成功サンプル1</title>
7:</head>
8:<body>
```

```
 9:     <h1>ログイン成功</h1>
10:     社員番号  : <span th:text = "${user.id}"></span><br>
11:     パスワード : <span th:text = "${user.password}"></span>
12:</body>
13:</html>
```

Thymeleaf の **th:text 属性は、指定された変数の値をHTML タグのテキスト内容として埋め込む**ために使用されます。

変数式は${...}という形で記述され、スコープ内にある変数の値を参照するために使われます。変数名は大文字・小文字を区別するため、Model に追加されたキー名と正確に一致する必要があります。

- **2 行目 <html xmlns:th="http://www.thymeleaf.org">**

10 行と、11 行には、th:text というthymeleaf 固有の属性が登場していますが、そのためには、予め宣言をする必要があります。

- **10 行目 **

この部分はThymeleaf 独自のスクレプト文法の一つの、th:text 属性を使用しています。

Controller からModel に格納した「user」というキーの後ろに、「.フィールド名」を記載することで、id の値を出力できます。例えば、user.id が1234 であれば、出力は1234となります。

【参考】 JSP でデータを表示する出力するコード

```
<%
        int id = (Integer) request.getAttribute("id");
        String password = (String) request.getAttribute("password");
 %>
    <p>社員番号 : <%= id %></p>
    <p>パスワード: <%= password %></p>
```

JSP では、上記のようにrequest.getAttribute の記述と、<% %>という複雑な書き方をする必要がありましたが、th:text を使うことで、それがかなり簡略化されました。

- **11 行目**

ここでもth:text属性を使い、userオブジェクトのpasswordフィールドの値を表示しています。変数式を使用することで、動的に内容をWeb ページに埋め込むことができます。

動作確認

http://localhost:8080/login3社員番号と、パスワードを入力してください。

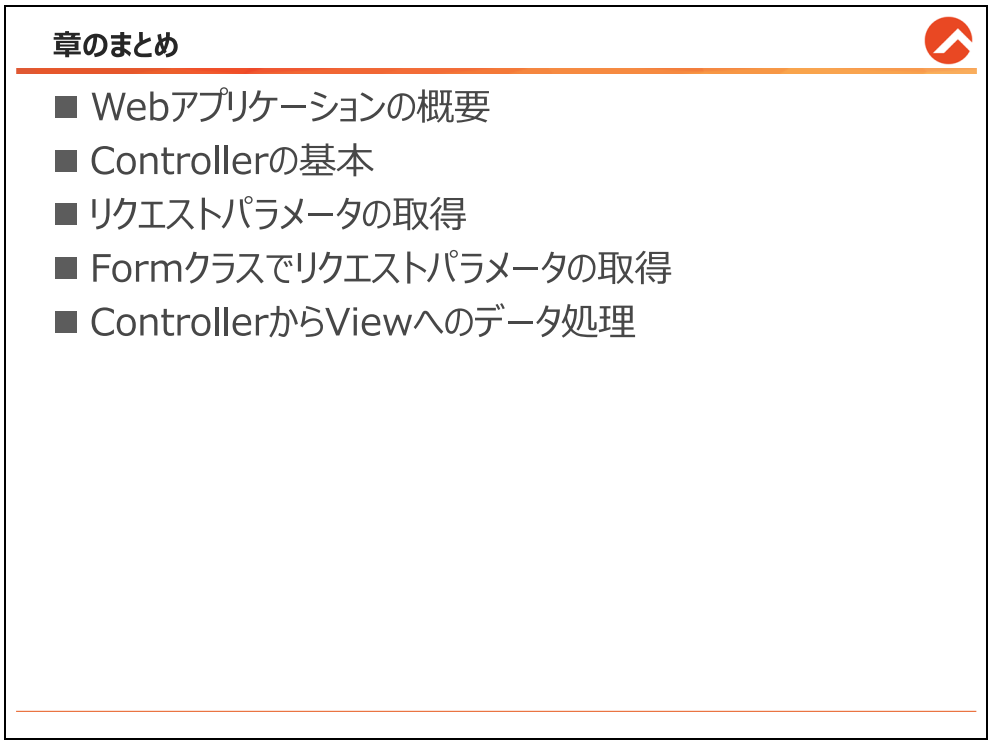


このようにThymeleaf テンプレートを使用すると、モデルに格納された属性をHTML ページ内で簡単に参照し、出力することができます。

別の章で紹介しますが、th:text の他にもThymeleaf は条件分岐、ループ処理、属性の変更など、多くの便利な機能を提供しています。

これにより、サーバーサイド(Controller)のロジックとView(HTML)の表示を分離しながら、動的なWeb ページを作成することが可能になります。

章のまとめ



この章では、Controller と View の作成方法と、データの受け渡しについてご説明しました。